

Importing Data Using Transform Maps

Spreadsheet to ServiceNow Table | Project Report

Platform: ServiceNow

Team Members: N.logeswari , S.christiena

Institution / Course: [MNM jain engineering college]

Date: [04 Oct 2026]

1. INTRODUCTION

1.1 Project Overview

Organisations keep large amounts of operational data in spreadsheets: employee lists, asset inventories, customer contacts and similar records. Entering these rows into ServiceNow one by one is slow and error-prone. This project demonstrates a reliable, repeatable way to bring spreadsheet data into ServiceNow using the platform's **Import Set** and **Transform Map** features.

An Excel file is attached to a **Data Source**, loaded into a temporary **Import Set (staging) table**, and then moved into a real **target table** such as User [sys_user] by a **Transform Map**. The transform map defines which spreadsheet column feeds which target field, how duplicates are detected (coalesce), and any scripted clean-up rules.

1.2 Purpose

- Replace manual data entry with an automated, auditable import process.
- Show every stage of the import: preparation, data source, load, transform map, field mapping, transform run and verification.
- Prevent duplicate and incomplete records by using coalesce fields and transform scripts.
- Provide a reusable template that other teams can follow for any spreadsheet-based import.

2. IDEATION PHASE

2.1 Problem Statement

ServiceNow administrators regularly receive spreadsheets that must be loaded into platform tables. Doing this manually causes typing mistakes, duplicate records, inconsistent formatting and wasted time, and there is no clear audit trail of what was imported.

Problem Statement	I am	I'm trying to	But	Because	Which makes me feel
PS-1	A ServiceNow administrator	Load hundreds of user records from a spreadsheet	Manual entry is slow and creates duplicates	Fields are not mapped and there is no duplicate check	Frustrated and uncertain about data quality
PS-2	A data owner / HR coordinator	Keep user data current after every hiring batch	I cannot tell which rows failed or were skipped	Import results are not reviewed systematically	Anxious about incorrect records

2.2 Empathy Map Canvas

The empathy map below captures the perspective of the primary user, a ServiceNow administrator who must import spreadsheet data.

Quadrant	What the user experiences
Says	"I have 500 rows to add before Monday." "Why did it create duplicates again?"
Thinks	"There must be a faster way than typing each record." "Will the import damage existing data?"
Does	Copies values by hand, checks records one at a time, asks colleagues how the import works.
Feels	Overloaded by repetitive work, worried about mistakes, unsure of platform features.
Pains	Duplicates, wrong field values, no error visibility, inconsistent spreadsheet formats.
Gains	Fast bulk loading, automatic duplicate handling, clear success/error report, repeatable process.

2.3 Brainstorming

The team listed possible approaches and scored them for effort and benefit.

Idea	Description	Effort	Benefit	Decision
Manual entry	Type each record into the form	High	Low	Rejected
List-view paste	Copy and paste into list editor	Medium	Low	Rejected

Idea	Description	Effort	Benefit	Decision
Import set + transform map	Load spreadsheet to staging table and transform	Low	High	Selected
Scheduled / REST import	Automate imports from files or APIs	Medium	High	Future scope
Flow Designer integration	Trigger imports from workflows	Medium	Medium	Future scope

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User action	Thinking / feeling	Pain point	Opportunity
Prepare	Cleans spreadsheet, fixes headers	Hopeful	Merged cells, blank rows	Provide a preparation checklist
Create data source	Defines Excel file source	Slightly unsure	Many settings to choose	Step-by-step guidance
Load data	Clicks Load All Records	Curious	Unsure rows loaded correctly	Review staging table first
Map fields	Creates transform map and field maps	Focused	Choosing the coalesce field	Auto Map + Mapping Assist
Transform	Runs the transform	Nervous	Fear of overwriting records	Test on small batch
Verify	Checks status and target records	Relieved	Understanding error rows	Use Transform History and error log

3.2 Solution Requirement

Functional requirements

ID	Requirement	Priority
FR-1	Accept spreadsheet data in Excel (.xlsx) or CSV format	High
FR-2	Create a data source that reads the sheet and header row	High
FR-3	Load all rows into an import set (staging) table	High
FR-4	Create a transform map from the staging table to the target table	High
FR-5	Map source fields to target fields, using Auto Map and Mapping Assist	High
FR-6	Use a coalesce field to update existing records and avoid duplicates	High
FR-7	Support transform scripts to skip or clean incomplete rows	Medium
FR-8	Show per-row status (inserted, updated, ignored, error) after the transform	High

Non-functional requirements

ID	Requirement	Description
NFR-1	Accuracy	Imported values must match the spreadsheet exactly
NFR-2	Auditability	Every import must be traceable through Transform History and logs
NFR-3	Security	Only users with import_admin or admin roles may run imports

ID	Requirement	Description
NFR-4	Reusability	The same map can be reused for future spreadsheets with the same layout
NFR-5	Usability	Steps must be simple enough for a new administrator to follow

3.3 Data Flow Diagram

The diagram shows how data moves from the administrator to the final table and how results flow back for verification.

Data Flow Diagram - Spreadsheet Import with Transform Map

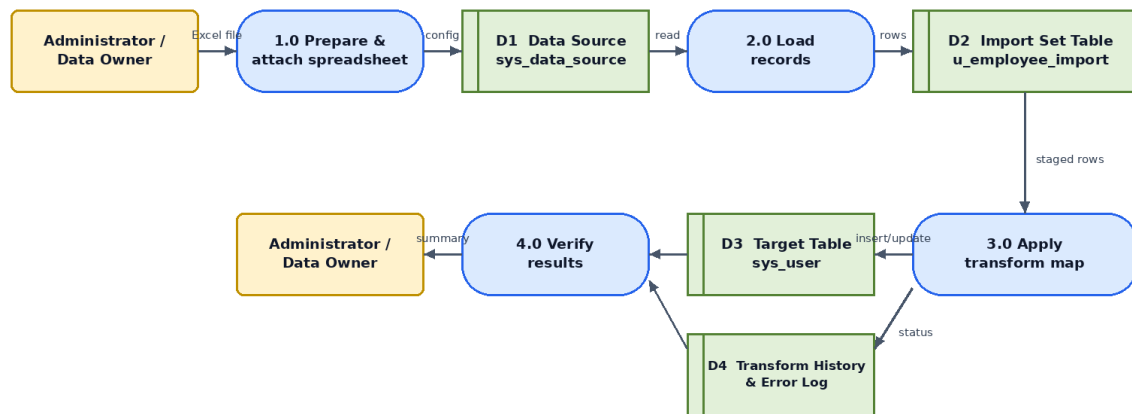


Figure 1: Data Flow Diagram

3.4 Technology Stack

Layer	Technology	Purpose
Platform	ServiceNow (Personal Developer Instance)	Hosts import sets, transform maps and target tables
Source data	Microsoft Excel (.xlsx) / CSV	Holds the records to be imported
Import tools	Data Source, Import Set table, Transform Map, Field Maps	Core import mechanism
Scripting	Server-side JavaScript (Glide) transform scripts	Row-level validation and clean-up
Target table	User [sys_user]	Receives the transformed records
Monitoring	Transform History, Import Log, Transform Errors	Verify and troubleshoot imports

4. PROJECT DESIGN

4.1 Problem Solution Fit

Aspect	Description
Customer segment	ServiceNow administrators, HR and IT operations teams who receive data in spreadsheets
Customer problem	Slow, error-prone manual entry with duplicates and no audit trail
Existing alternatives	Manual entry, list editor paste, ad hoc scripts
Our solution	Standardised import using a data source, staging table and transform map with coalesce
Why it fits	Uses built-in platform features, needs no extra software, and is repeatable and auditable
Triggers to act	New hire batches, asset audits, data clean-up projects
Outcome	Faster loading, fewer duplicates, clear success/error reporting

4.2 Proposed Solution

Parameter	Description
Problem being solved	Bulk loading of spreadsheet data into ServiceNow without duplicates or manual typing
Idea / solution	Use Import Sets with a Transform Map: spreadsheet, data source, staging table, transform map, target table
Novelty	A complete, documented walkthrough with coalesce and an onBefore script for data quality
Social impact	Saves administrator time and improves the accuracy of organisational data
Business model	Reusable internal process template for any department importing spreadsheets
Scalability	The same pattern extends to scheduled, REST or large-file imports

4.3 Solution Architecture

The architecture has three zones. The **source** zone holds the cleaned spreadsheet. The **ServiceNow platform** zone contains the data source, the import set table and the transform map with its field maps and scripts. The **output** zone contains the target table records plus the transform history and error logs used for verification.

Solution Architecture - Transform Map Import

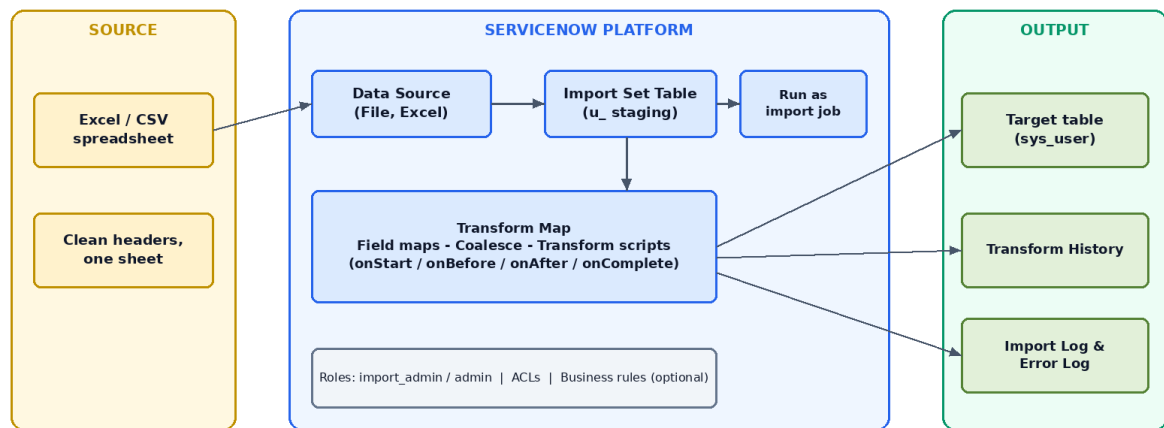


Figure 2: Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Task	Owner	Duration	Status
1. Research	Study import sets, data sources and transform maps	[Member 1]	Week 1	[Planned / Done]
2. Preparation	Prepare sample spreadsheet and clean data	[Member 2]	Week 1	[Planned / Done]
3. Configuration	Create data source and load staging table	[Member 1]	Week 2	[Planned / Done]
4. Mapping	Create transform map, field maps and coalesce	[Member 3]	Week 2	[Planned / Done]
5. Scripting	Write and test onBefore transform script	[Member 2]	Week 3	[Planned / Done]
6. Testing	Run transform, verify results, fix errors	[Member 3]	Week 3	[Planned / Done]
7. Documentation	Prepare report, screenshots and demo video	[All]	Week 4	[Planned / Done]

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Testing covered both functional behaviour and import performance. Record the actual result for each case after running it on your instance.

Functional test cases

ID	Scenario	Input	Expected result	Actual / Status
TC-01	Load valid spreadsheet	10 valid rows	10 rows appear in the import set table	[Fill after test]
TC-02	Transform new records	10 new emails	10 records inserted into sys_user	[Fill after test]
TC-03	Re-import same file	Same 10 rows	Rows updated, no duplicates (coalesce on email)	[Fill after test]
TC-04	Row with empty email	1 row without email	Row ignored by onBefore script	[Fill after test]
TC-05	Invalid reference value	Unknown department	Row handled per field map; error visible in log	[Fill after test]
TC-06	Wrong sheet / header row	Header on row 2	Columns misread; fix data source settings	[Fill after test]

Performance observations

Metric	Test batch	Measured value	Notes
Load time	10 rows	[] seconds	Data source to staging table
Load time	500 rows	[] seconds	Data source to staging table
Transform time	500 rows	[] seconds	With and without business rules
Error rows	500 rows	[]	Reviewed in Transform Errors

7. RESULTS

7.1 Output Screenshots

Paste the screenshots from your own instance below. They are listed in the order of the import process.

employees.xlsx - Excel

	A	B	C	D	E	F
1	u_first_name	u_last_name	u_email	u_department	u_phone	u_title
2	Asha	Kumar	asha.kumar@example.com	IT	98400 10001	Analyst
3	Ravi	Menon	ravi.menon@example.com	HR	98400 10002	Coordinator
4	Meena	Iyer	meena.iyer@example.com	Finance	98400 10003	Accountant
5	Karthik	Raj	karthik.raj@example.com	IT	98400 10004	Developer
6	Divya	Nair	divya.nair@example.com	Support	98400 10005	Agent

Sheet1 | header in row 1, data on a single sheet

Illustrative example - Team ID: SWTID-2026-5741

Figure 3: Source spreadsheet with header row

servicenow Search dev000000.service-now.com | System Administrator

Data Source - Employee Import Data Source System Import Sets > Administration > Data Sources

Name Employee Import Data Source	Import set table label Employee Import
Import set table name u_employee_import	Type File
Format Excel	File retrieval method Attachment
Sheet number 1	Header row 1
Attachment employees.xlsx (12 KB)	

Update **Load All Records**

Illustrative example - Team ID: SWTID-2026-5741

Figure 4: Data Source form (Type: File, Format: Excel)

servicenow

Search

dev000000.service-now.com | System Administrator

Employee Import [u_employee_import]

System Import Sets > Import Sets > Employee Import

Number	First name	Last name	Email	Department	Phone
IMP0001001	Asha	Kumar	asha.kumar@example.com	IT	98400 10001
IMP0001002	Ravi	Menon	ravi.menon@example.com	HR	98400 10002
IMP0001003	Meena	Iyer	meena.iyer@example.com	Finance	98400 10003
IMP0001004	Karthik	Raj	karthik.raj@example.com	IT	98400 10004
IMP0001005	Divya	Nair	divya.nair@example.com	Support	98400 10005

1 to 5 of 5 rows

State: Loaded | Import set: ISET0010001 | Related link: Transform (choose Employee Transform)

Illustrative example - Team ID: SWTID-2026-S741

Figure 5: Import set table after Load All Records

servicenow

Search

dev000000.servicenow.com | System Administrator

Table Transform Map - Employee Transform

System Import Sets > Administration > Transform Maps

Name

Employee Transform

Order

100

Source table

Employee Import [u_employee_import]

Target table

User [sys_user]

Active

☒ true

Run business rules

☒ true

Enforce mandatory fields

No

Copy empty fields

☐ false

Update

Auto Map Matching Fields

Mapping Assist

Related Lists: [Field Maps](#) | [Transform Scripts](#) | [Entity Transforms](#)

Illustrative example - Team ID: SWTID-2026-5741

Figure 6: Transform Map form (source and target tables)

servicenow

Search

dev000000.service-now.com | System Administrator

Field Maps - Employee Transform

System Import Sets > Administration > Transform Maps > Field Maps

Source field	Target field	Coalesce	Use source script
u_first_name	first_name	false	false
u_last_name	last_name	false	false
u_email	email	true	false
u_department	department	false	false
u_phone	phone	false	false
u_title	title	false	false

Highlighted row: Coalesce = true on the email field, so an existing user with the same email is updated instead of duplicated.

Illustrative example - Team ID: SWTID-2026-5741

Figure 7: Field maps with coalesce on email

servicenow

Search

dev000000.service-now.com | System Administrator

Transform History - Employee Transform

System Import Sets > Transform History

Number	Transform map	State	Inserted	Updated	Ignored	Errors
TMH0001001	Employee Transform	Complete	5	0	0	0

Second run after adding one row with a blank email:

Number	Transform map	State	Inserted	Updated	Ignored	Errors
TMH0001002	Employee Transform	Complete	0	5	1	0

Re-running the same data updates the records (coalesce on email) and the incomplete row is ignored by the onBefore script.

Illustrative example - Team ID: SWTID-2026-5741

Figure 8: Transform History showing completed run

servicenow Search dev000000.service-now.com System Administrator																																		
Users [sys_user] User Administration > Users																																		
<table> <tr> <th>User ID</th><th>Name</th><th>Email</th><th>Department</th><th>Title</th></tr> <tr> <td>asha.kumar</td><td>Asha Kumar</td><td>asha.kumar@example.com</td><td>IT</td><td>Analyst</td></tr> <tr> <td>ravi.menon</td><td>Ravi Menon</td><td>ravi.menon@example.com</td><td>HR</td><td>Coordinator</td></tr> <tr> <td>meena.iyer</td><td>Meena Iyer</td><td>meena.iyer@example.com</td><td>Finance</td><td>Accountant</td></tr> <tr> <td>karthik.raj</td><td>Karthik Raj</td><td>karthik.raj@example.com</td><td>IT</td><td>Developer</td></tr> <tr> <td>divya.nair</td><td>Divya Nair</td><td>divya.nair@example.com</td><td>Support</td><td>Agent</td></tr> </table>					User ID	Name	Email	Department	Title	asha.kumar	Asha Kumar	asha.kumar@example.com	IT	Analyst	ravi.menon	Ravi Menon	ravi.menon@example.com	HR	Coordinator	meena.iyer	Meena Iyer	meena.iyer@example.com	Finance	Accountant	karthik.raj	Karthik Raj	karthik.raj@example.com	IT	Developer	divya.nair	Divya Nair	divya.nair@example.com	Support	Agent
User ID	Name	Email	Department	Title																														
asha.kumar	Asha Kumar	asha.kumar@example.com	IT	Analyst																														
ravi.menon	Ravi Menon	ravi.menon@example.com	HR	Coordinator																														
meena.iyer	Meena Iyer	meena.iyer@example.com	Finance	Accountant																														
karthik.raj	Karthik Raj	karthik.raj@example.com	IT	Developer																														
divya.nair	Divya Nair	divya.nair@example.com	Support	Agent																														
1 to 5 of 5 rows (filtered: Created today)																																		
Illustrative example - Team ID: SWTID-2026-5741																																		

Figure 9: Target table (sys_user) with imported records

8. ADVANTAGES & DISADVANTAGES

Advantages

- Imports hundreds of rows in one operation instead of manual entry.
- Staging table lets you review data before it reaches real tables.
- Coalesce updates existing records and prevents duplicates.
- Transform scripts enforce data quality rules.
- Built-in history and logs make every import auditable.
- No extra tools or licences are needed beyond the platform.

Disadvantages

- Spreadsheet must be well formatted; messy files cause errors.
- Initial setup of data source and field maps takes time.
- Reference fields (such as department) need matching values or extra mapping logic.
- Poorly chosen coalesce fields can overwrite the wrong records.
- Large imports can load the instance and should be scheduled carefully.

9. CONCLUSION

The project shows that spreadsheet data can be imported into ServiceNow safely and repeatably using a data source, an import set table and a transform map. Staging the data first, mapping fields explicitly, using a coalesce field and adding a small transform script give accurate results with a full audit trail. The same process can be reused for many other tables and teams.

10. FUTURE SCOPE

- Schedule recurring imports from an SFTP or MID Server file location.
- Load data through REST/JSON sources and the Import Set API.
- Trigger imports and notifications from Flow Designer.
- Add automated validation rules and error-notification emails.
- Build a self-service portal widget for non-admin uploads.
- Extend the pattern to other tables such as assets, locations and groups.

11. APPENDIX

Source Code (if any)

Sample transform script (onBefore) that skips rows without an email address:

```
// Transform Map > Transform Scripts > onBefore
(function runTransformScript(source, map, log, target) {

    // Skip incomplete rows
    if (source.u_email.nil()) {
        ignore = true;
        log.info('Row skipped: no email address');
    }

})(source, map, log, target);
```

Sample spreadsheet layout (row 1 = header):

u_first_name	u_last_name	u_email	u_department	u_phone	u_title
Asha	Kumar	asha.kumar@example.com	IT	9876500001	Analyst
Ravi	Menon	ravi.menon@example.com	HR	9876500002	Coordinator

Note: On the User table, the Name field is calculated from First name and Last name, so map those two fields rather than Name.

Dataset Link

[Paste the link to your sample spreadsheet / Google Drive here]

GitHub & Project Demo Link

GitHub: [Paste repository link here]

Demo video: [Paste demo video link here] (file: Import_Data_Using_Transform_Map.mp4)